SYNSEMA: Syntax-Guided Learning of Semantically Valid Programs

Levin Winter **Cong Li** **Hao Sun** **Zhendong Su**
ETH Zurich, Switzerland
{levin.winter, cong.li, hao.sun, zhendong.su}@inf.ethz.ch

Abstract

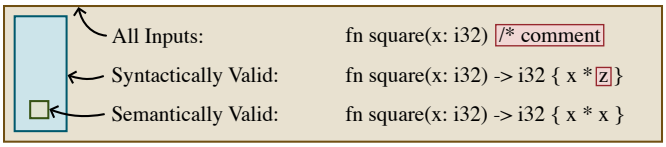
Generating effective inputs for testing systems that process highly structured data, such as compilers, is difficult due to the combination of syntactic and semantic constraints. Traditional approaches rely on manually crafted rules that require deep domain expertise and are difficult to maintain, while modern machine learning methods have to learn syntax and semantics simultaneously, which is challenging. We present SYNSEMA, a novel approach that combines insights from both worlds by reformulating the generation of semantically valid programs as a syntax-guided reinforcement learning task. With syntax guaranteed by construction, even a small, randomly initialized model can rapidly learn semantic constraints in a drastically reduced space, enabling effective compiler testing. We implement our approach in the context of Java and Rust, which both exhibit strict semantic requirements, and train an agent to generate interesting and well-formed programs using fine-grained semantic feedback. Applied to compiler testing, we systematically compare different syntax-guided generation procedures with an unconstrained baseline and show that, by restricting the sampling space, SYNSEMA learns semantic rules efficiently. While an average of only $0.03\,\%$ of programs generated by the baseline are semantically valid, SYNSEMA produces a diverse set of programs with a $7.29\,\%$ success rate that trigger $2.6\times$ more compiler behaviors. In addition, SYNSEMA discovered 5 previously unknown bugs in production compilers, validating the practical impact of our approach.

1 Introduction

Effective compiler testing requires diverse, *well-formed* programs. However, generating these programs is challenging because they have to fulfill dual constraints: syntax and semantics. The difficulty comes from the vast space of programs, which consists of all possible combinations of valid characters with unbounded possible length, yet only a tiny fraction of them are both syntactically and semantically valid. As illustrated in Figure 1a, the programs of interest must first satisfy the grammar, *e.g.*, a valid derivation of the production rules (Section 2), and more importantly, fulfill the semantic rules, *e.g.*, a valid inference chain of type rules exists for the program. Each layer narrows the space, thereby significantly increasing the difficulty of generating one input in that space.

Traditionally, this is handled by crafting intricate program generators [8; 27; 33; 37], *e.g.*, by deriving programs from a large set of grammar rules while simultaneously collecting the program state for the partially generated program and using it to solve the semantic constraints. This manual process of encoding syntax and semantic rules normally follows the language specification, which requires deep domain expertise and significant effort due to the sheer size of the specification (*e.g.*, the 800-page Java Language Specification [11]). Recently, autoregressive language models automate this process: a model is trained on token sequences and over time learns to reproduce both the correct syntax and semantics. However, learning syntax and semantics simultaneously is challenging and

40th Conference on Neural Information Processing Systems (NeurIPS 2026).



(a) Semantically valid inputs are rare (red boxes indicate errors).

Expr	$E \rightarrow E\ O\ E$	(i)
Expr	$E \rightarrow$ `NUM`	(ii)
Op	$O \rightarrow$ `+`	(iii)
Op	$O \rightarrow$ `*`	(iv)

(b) A minimal example CFG

Figure 1: Illustration of Input Space and Context-Free Grammar (CFG)

inefficient [1], as syntactically invalid programs constitute the overwhelming majority of the token space [31]. As shown in Figure 1a, the model must first learn to avoid generating inputs in the enormous outermost space (*i.e.*, "All Inputs"); then shift its focus to the inner space of interest (*i.e.*, "Semantically Valid"), leading to a high early-rejection rate at training time: more than $99.9\,\%$, as indicated by our experiments.

SYNSEMA. Our insight is a separation of concerns. We encode syntax rules to ensure that syntactically invalid programs are avoided during training *by construction*, allowing the models to learn semantics in a much smaller space. This approach requires minimal human effort and domain expertise, since context-free grammars (CFGs) are readily available for most production languages [3; 22] and the relevant algorithms are well-studied in the literature [2; 9; 16]. Next, we reformulate program generation for compiler testing as a *syntax-guided reinforcement learning (RL) task*, by leveraging CFGs to constrain the action space. We design reward functions to guide the RL agent (or *agent*) to learn semantic correctness; reward functions can also be extended for specific user concerns. Together, this yields an automated, efficient learning-based generator of programs of interest.

Our implementation[1] uses CFGs to derive a small subset of syntactically valid actions based on the generated program prefix, which can be represented as either (1) valid language tokens, *e.g.*, the `let` keyword in Rust, or (2) parsing operations such as `reduce` : $E \rightarrow$ `NUM` (reduces a number into an expression). We systematically explore three types of well-studied parsing algorithms to constrain the action space: LL as a top-down approach, LR as a bottom-up approach, and Earley parsing as a chart-based approach. For semantics (and interestingness), the compiler naturally acts as an automated oracle that evaluates programs and returns rich feedback, which we use as reward signal.

Contributions. We show that, with syntax guaranteed by construction, a small, randomly initialized model can learn semantic constraints for effective compiler testing. We contribute:

- A novel approach to compiler testing that separates syntax from learning semantics by combining parser-derived action spaces with compiler-feedback RL. We conduct a systematic exploration of the design space for syntax-guided reinforcement learning environments.
- An implementation of SYNSEMA and, to our knowledge, the first systematic evaluation of top-down, bottom-up, and chart parsing as RL action spaces for learning semantic constraints in compiler testing, compared with a token-based baseline.
- SYNSEMA produces $7.29\,\%$ compilable, semantically valid programs vs. $0.03\,\%$ baseline, achieves $2.6\times$ higher source coverage and finds five new compiler bugs using syntax guidance. We also compare against the state-of-the-art LLM-based fuzzer Fuzz4All using DeepSeek and find substantially more Rust compiler bugs (Section 4.2).

2 Background

Context-free grammars. A CFG is formally represented as a 4-tuple $G = (N, T, P, S)$ [13], defined by a nonterminal set N, a terminal set T, a production set P, and a start symbol S. To illustrate, we devised a small grammar in Figure 1b for simple arithmetic expressions and use it as an example throughout the paper. In this example, there are three terminals (`NUM`, `+`, and `*`) and two nonterminals (Op and $Expr$). The core of the grammar is its set of productions P, *e.g.*, (i) to (iv), where each denotes a substitution rule by which any occurrence of the nonterminal on its left-hand side could be replaced by the right-hand side. For example, the first rule states that any occurrence of an expression E can be replaced with the sequence $E\ O\ E$. A syntactically valid program following this CFG is `NUM+NUM*NUM`; in contrast, `NUM-NUM,NUM` is invalid.

[1] Code: https://github.com/levinwinter/synsema; data: https://doi.org/10.5281/zenodo.20339950.

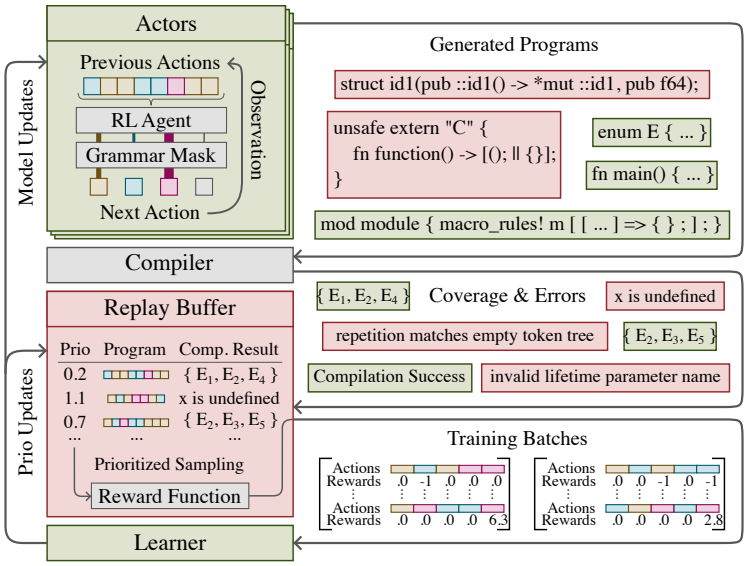


Figure 2: SYNSEMA consists of four components: actors, the compiler, replay buffer, and learner.

Parsing performs syntactic analysis by transforming unstructured input such as the aforementioned `NUM+NUM*NUM` into a structured representation such as `+(NUM *(NUM NUM))` according to the CFG. For an invalid input such as `NUM-NUM,NUM`, parsing reports a syntax error, and no such structured representations can be established. Three major types of algorithms exist in the literature:

- **Top-down parsing**, *e.g.*, the LL algorithm [2], begins with the start symbol and repeatedly applies productions to the current leftmost nonterminal until it reconstructs the input. Using the generated sequence of productions, it builds a top-down parse tree from the root toward the leaves.
- **Bottom-up parsing**, exemplified by the LR algorithm [16], starts with the input terminals and reduces them back to the start symbol. Guided by a state machine derived from the grammar (Table 5 illustrates an example), it will either *shift* a terminal onto its parsing stack or *reduce* the top of the stack that matches a production's right-hand side to its left-hand side nonterminal.
- **Chart parsing**, like the Earley parsing [9], is a chart-based algorithm that employs dynamic programming to process any context-free grammar. For each input position it tracks all possible parse trees simultaneously. By systematically applying prediction, scanning, and completion steps, the algorithm identifies all valid derivations without the need for backtracking.

Reinforcement learning. RL trains an agent that interacts with an environment to maximize a reward signal [15]. At every timestep, the agent selects an action based on its current observation of the environment and receives a reward for the action and a new observation. The goal of the agent is to maximize the cumulative, discounted reward over time. To this end, we aim to learn a policy π mapping the sequence of past observations to the optimal action.

3 Approach

We present SYNSEMA, a novel approach that reformulates input generation for compiler testing as a syntax-guided reinforcement learning task. Our key insight is to separate concerns between syntax and semantics by specifying the former and learning the latter. We harness the readily available grammars to ensure syntactic validity and allow the agent to learn semantic constraints through fine-grained compiler feedback. Here, semantic validity means conformance to the language's semantic constraints, such as typing, scope, and ownership rules. We use compiler acceptance as a proxy for semantic validity in training and evaluation. Its RL environment has four components (Figure 2):

- *Actor* (Section 3.1). The actor encodes the CFG into a constrained action space from which the RL agent selects the next syntactically valid action given the current state, *i.e.*, the previously generated action sequence. Depending on the action representation, the actor may also transform

Table 1: An overview of the three syntax-guided RL agents with a visualization of their action space and respective representations. Productions are marked red and terminals green.

Agent	Actions	Action Sequence (Example)
Top-Down	Productions	*E→E O E* | *E→NUM* | *O→+* | *E→E O E* | *E→NUM* | *O→✻* | *E→NUM*
Bottom-Up	Terminals & Productions	NUM | *E→NUM* | + | *O→+* | NUM | *E→NUM* | ✻ | *O→✻* | NUM | *E→NUM* | *E→E O E* | *E→E O E*
Chart	Terminals	NUM | + | NUM | ✻ | NUM

the current action sequence into a program. This design mitigates early compilation failures due to syntax errors by ensuring that all generated programs are syntactically well-formed.

- *Compiler*. The compiler processes the program and returns targeted feedback, including compilation status, error messages, and coverage information as an indicator of the program's uniqueness.
- *Replay Buffer* (Section 3.2). The replay buffer uses compiler feedback to assign rewards to past actions. When a generated program fails to compile, it traces the error messages back to the specific agent decisions that triggered them and assigns penalties accordingly. In addition, a coverage-based reward encourages the generation of diverse and interesting programs, guiding the agent toward underexplored compiler behaviors.
- *Learner*. The learner computes gradients and updates the RL agent's weights for the next timestep.

3.1 Action Space Encodes Syntax Rules

The actor leverages the CFG to derive a small subset of syntactically valid actions conditioned on the current action sequence. We consider two action representations and instantiate them with three well-established parsing algorithms, namely, LL for top-down parsing, LR for bottom-up parsing, and Earley for chart parsing, each inducing a different constrained action space and implicating a different information flow direction; we explore each design choice. We present the parser-specific algorithms used by the actor for program generation in Appendix A.1. Worked examples and alternative representations are discussed in Appendix A.2.

Action representation. An action in SYNSEMA can be represented in two ways: (1) as a valid language token, *e.g.*, the `let` keyword in Rust; or (2) as a parsing operation, *e.g.*, `reduce` : $E \rightarrow$ `NUM`, which reduces a number to an expression in an LR parsing algorithm. We refer to these as the *token-based* and *operation-based* representations, respectively. Under the token-based representation, the action sequence directly forms a program prefix. Under the operation-based representation, the actor translates the sequence of parser-specific operations into the program prefix.

Top-down parsing. We use an operation-based representation for the top-down parser that implements a canonical LL algorithm. The agent starts from the start symbol and, at each step, expands the leftmost nonterminal by choosing a valid production rule. Once no nonterminals remain, the program is fully generated and the procedure terminates. Under this representation, each action corresponds to selecting a production rule, and the full action space is the set of all production rules in the CFG. However, at any given step, the actor parses the current action sequence to identify the leftmost nonterminal and constrains the action space so that *only* productions whose left-hand side matches that nonterminal are valid.

Bottom-up parsing. We adopt an operation-based representation for the bottom-up parser which leverages the LR parsing algorithm by mimicking a shift-reduce parser. The action space consists of a `shift` : $terminal$ operation for every terminal and a `reduce` : $production$ operation for every production of every nonterminal in the CFG. Shift operations generate new terminals which extend the program under generation, while reduce operations apply productions to collapse sequences of terminals and nonterminals into nonterminals, eventually reducing to the start symbol that indicates the completion of a program. The agent traverses the LR automaton derived from the CFG to determine which shift/reduce actions are valid in the current state.

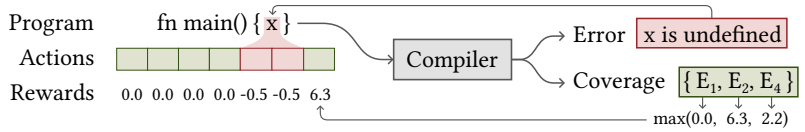


Figure 3: Reward calculation for a fictitious program. A negative reward maps to the offending action due to a compilation failure, whereas a compiler coverage-based novelty score is assigned to the final action for a semantically valid program.

Chart parsing. Unlike the other parsers, the chart parser based on the Earley algorithm uses a token-based representation. Given the current program prefix, the agent applies Earley to derive the set of syntactically valid terminals that can legally appear next.

3.2 Reward Function Models Semantic Rules

We aim to generate semantically valid, diverse, and interesting programs. Therefore, our strategy is twofold: we penalize semantic errors and encourage programs that trigger unobserved behavior.

Compilation feedback. To incentivize the generation of semantically valid programs, we provide fine-grained compilation feedback. The key insight is that compilation errors (*i.e.*, semantic errors such as the use of undefined variables, since the program is syntactically valid by construction) can be jointly caused by multiple actions. Thus, when compilation errors occur, we recognize and penalize the offending ones. We leverage the source spans that the compiler attaches to error messages. As shown in Figure 3, these spans mark the code regions that the compiler believes to be faulty and are typically printed to help developers pinpoint error locations. We map these locations back to the corresponding agent actions and assign negative rewards to each of them. This token-to-action mapping is constructed by the actors when translating the action sequence into the program. Because multiple errors may occur at the same time, for each error, we uniformly distribute a penalty of -1 over all actions that contributed to the respective error span. This approach incentivizes the agent to minimize compilation errors.

Coverage feedback. We discover new compiler behaviors by monitoring its code coverage. In particular, we encourage programs that execute previously uncovered code regions, as these exercise unseen compiler behaviors and promote more diverse and interesting program generation. Our reward mechanism draws inspiration from traditional coverage-guided fuzzing methods. For each program, we measure the compiler's edge coverage by tracking unique transitions between basic block pairs. For Java, we approximate this coverage using standard techniques, as detailed in Section 4.2. For each edge, we calculate a novelty score based on how many unique programs have exercised that edge. When an edge is first discovered, its novelty score is 10. As more unique programs cover the same edge, the score decreases linearly. Once $1{,}000$ unique programs have exercised the same edge, the score decays to 0. If the program compiles successfully, we assign a reward to the final action equal to the maximum novelty score across all edges that the program covered. Figure 3 illustrates this reward assignment: for a program that exercises three edges with novelty scores of 0.0, 6.3, and 2.2, the final action receives a reward of 6.3, *i.e.*, the highest among the covered edges.

3.3 Implementation Details

We employ Ape-X [14] and adopt deep Q-learning to demonstrate SYNSEMA by generating small programs for compiler testing. Rather than sequentially generating programs, calculating rewards, and updating the model in a single loop, our system performs these operations concurrently. As shown in Figure 2, SYNSEMA consists of a single learner responsible for weight updates and multiple concurrent actors to generate programs. Communication between components occurs asynchronously through the replay buffer that stores all generated programs and enables prioritized sampling of training batches for the learner. Actors restrict action selection to syntactically valid actions. The learner applies the same constraints when selecting next actions for its Q-learning targets, preventing value estimates from depending on unavailable actions.

Compilers generally require complete programs and cannot evaluate arbitrary prefixes. Therefore, the actor must finish generating a full program before it can be passed to the compiler. This can require an unpredictable number of steps for large programs. As a prototype to demonstrate the

Table 2: Effectiveness and success rate comparison for different agents. We report the total number of programs generated and the fraction of unique compilable programs. In addition, we measure the average number of steps required to generate one token.

	Agent	Total	Semantically Valid (min/median/max)				Steps/Token
Rust	Baseline	12,022,603	58	58	(0.01 %)	220	1.00
	Top-Down	6,331,849	373,891	493,432	(7.79 %)	528,945	1.49
	Bottom-Up	4,253,549	405,453	602,502	(14.16 %)	1,276,429	3.26
	Chart	7,772,170	566,768	700,826	(9.02 %)	851,096	1.00
Java	Baseline	954,065	370	438	(0.05 %)	509	1.00
	Top-Down	572,472	472	6,594	(1.15 %)	32,100	1.70
	Bottom-Up	877,948	44,528	46,578	(5.31 %)	88,731	2.64
	Chart	644,308	14,129	40,774	(6.33 %)	50,660	1.00

effectiveness of SYNSEMA, we cap each action sequence at 128 actions for small program generation. If the agent has not terminated generation by this point, we compute the shortest suffix that completes the current prefix into a syntactically valid program. This way, we avoid many early rejections due to non-termination. In addition, since terminals like identifiers and literals can induce huge action spaces, we treat them as nonterminals whose right-hand sides are tokens (a.k.a., subwords) which could be obtained with standard subword tokenization techniques such as BPE [25]. For our work, we restrict both to a predefined vocabulary, allowing them to efficiently be produced in one step.

4 Empirical Evaluation

To evaluate SYNSEMA, we design a baseline using token-based representations. The baseline is an RL agent that models how LLMs and other state-of-the-art techniques typically generate programs *without* separating the concern of syntax and semantics. The agent samples from the complete vocabulary, allowing it to choose any token at any time. To end the generation, the agent can select a special end-of-sequence action. The baseline has the same reward mechanisms (Section 3.2).

Result highlights. In brief, our evaluation shows the following:

1. 7.29 % of programs generated by SYNSEMA are accepted by the compiler, compared to the baseline which has an average success rate of 0.03 %.
2. The programs generated with syntax guidance exercise more diverse compiler behaviors as they result in 2.6× higher coverage and expose 5 new bugs in Rust's and Java's compilers.
3. Our fine-grained semantic feedback provides rich learning signals as it achieves 25 % higher edge coverage compared to agents trained without targeted error rewards.

Experimental setup. We target Java 21 featuring rich semantics and Rust, a challenging system to generate programs for [31] due to its complex type system, *e.g.*, strict ownership rules. The compilers are `javac` and `rustc`, two mature, production-grade compilers. We compare the baseline and three syntax-guided variants over three five-day runs each; plots show medians and min/max ranges. We employ the GPT-2 (124M) architecture, with the default of 12 layers, 12 attention heads, and an embedding dimension of 768, and train it from scratch with randomly initialized weights. All RL experiments are run on the Euler cluster at ETH Zurich with two NVIDIA GeForce RTX 4090 GPUs, 20 CPU cores, and 80 GB RAM each. The remaining settings (*e.g.*, learning rate schedule and optimizer) are available in Appendix A.4 and the code.

4.1 Effectiveness of SYNSEMA

We evaluate SYNSEMA's effectiveness at generating semantically valid programs, detailed in Table 2.

Total number of programs. We measure the median total number of programs generated during the course of five days. The baseline generated more than 12 million Rust and 950K Java programs compared to our syntax-guided variants, which generated 4.2 to 7.7 million Rust and 570K to 870K

Java programs. This slight lag arises because, in the absence of syntax guidance, the baseline rapidly generates many syntactically invalid programs. Compilers reject such programs quickly, as their middle and back ends are bypassed and only parts of the front end are exercised. This also speeds up coverage counting for reward computation and allows more programs to be generated. This effect is more pronounced for Rust than for Java: the startup overhead of `javac` is observed to be higher and dominates the overall time required, which in turn results in fewer Java programs being generated compared to Rust. Nevertheless, the consequent space reduction from SYNSEMA allows agents to produce far more semantically valid programs, as described next.

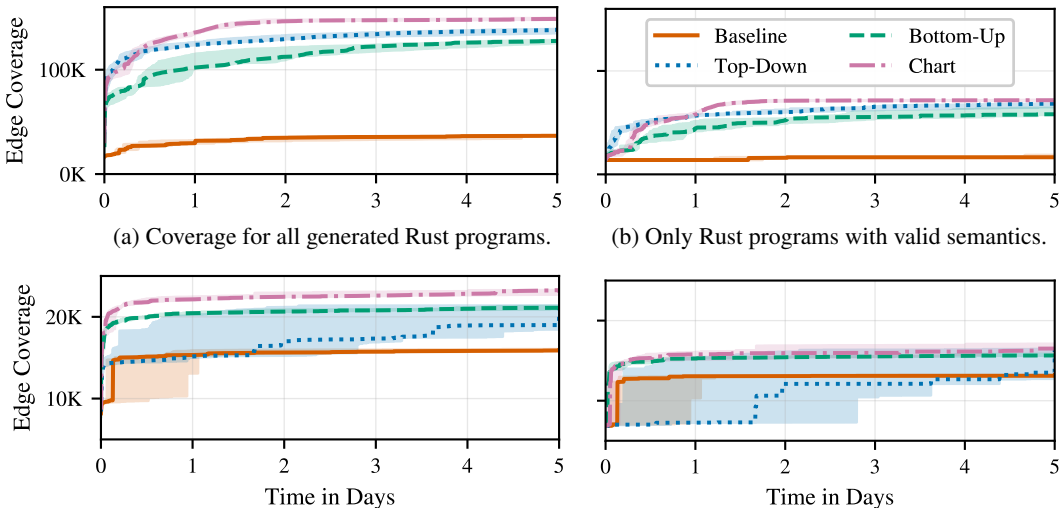
Semantic correctness. The effectiveness gap becomes stark when examining the number of unique programs that successfully compile, since they adhere to all semantic constraints encoded inside the respective compiler. At most $0.05\,\%$ of the unique programs generated by the baseline are semantically correct. SYNSEMA's syntax-guidance is much more effective since, on average, $7.29\,\%$ of programs could be compiled. The agent can directly learn semantically valid programs and does not suffer from the many early rejections due to incorrect syntax. To better understand the effects of the agent's learning, we compare SYNSEMA with random policies on Rust (Appendix A.5), which provides additional evidence that SYNSEMA learns to use nontrivial semantic features beyond syntax enforcement: learned policies exercise these features more frequently than random policies, with macros as the exception. We observe that in the case of Java, the agent based on the top-down parser is less effective than the other two. We analyzed the corresponding runs and found that this agent frequently failed to terminate generation successfully, often producing incomplete yet similar programs: all other agents typically generate programs in less than 93 steps on average, well below the threshold of 128 after which we take control from the agent and calculate the shortest syntactically valid completion, while for top-down agent, it took an average of 177.89 steps, hindering it from effectively learning valid semantics.

Representation overhead. The syntax-guided agents with operation-based representations (*i.e.*, top-down and bottom-up) can incur overhead by making the syntactic structure explicit and introducing additional actions, as can be observed in Table 1. We measure the average number of steps required to generate one token. Naturally, the baseline and the chart-based agents emit one token per step. The bottom-up agent has the highest average overhead of 2.95 steps per token because the agent needs to perform exactly one shift operation but also multiple reduce operations to merge tokens and subtrees into the final abstract syntax tree. The two top-down agents have less overhead at 1.49 and 1.70; they benefit from productions that can emit multiple terminals simultaneously. Despite this overhead, operation-based agents still generate $14\times$ to $10{,}387\times$ semantically valid programs; in this context, the bottom-up agent even outperforms the chart agent for Rust.

4.2 Utility of SYNSEMA

Next, we evaluate SYNSEMA's utility in terms of (1) program diversity, measured by the compiler coverage underlying our reward function, and (2) its ability to generate rare inputs that trigger unexpected compiler crashes.

Compiler behaviors. To assess program diversity, we measure compiler coverage (Section 3.2). In the case of Java, we approximate this coverage by hashing executed JaCoCo probes into a bitmap with $65{,}536$ entries, following the hashing approach used by fuzzers such as AFL [38]. Figure 4 shows the results: plots (a) and (c) on the left report coverage over all generated programs, whereas plots (b) and (d) on the right report coverage restricted to semantically valid programs only. They show that our syntax-guidance yields considerably higher diversity. The chart agent performs the best, with $1.45\times$ and $4.05\times$ higher coverage than the baseline for Java and Rust, respectively; for valid programs, the improvements are $1.26\times$ and $4.33\times$. For Rust, the top-down and bottom-up agents also exercise at least $3.48\times$ more behaviors. When considering Java, the bottom-up agent surpasses the baseline by $32.3\,\%$ in the general setting. However, in the case of semantically valid Java programs, the top-down agent performs only minimally better than the baseline. This shares the same root cause as previously described in Section 4.1: the agent frequently failed to terminate generation within its step-budget, which reduced its ability to generalize and led to high variance in learning. With this exception removed, SYNSEMA consistently outperforms the baseline in both setups. This demonstrates that SYNSEMA increases coverage over time not just through exploring error handling, but also through compilable, meaningful programs.



(a) Coverage for all generated Rust programs.

(b) Only Rust programs with valid semantics.

(c) Coverage for all generated Java programs.

(d) Only Java programs with valid semantics.

Figure 4: Coverage of the Rust and Java compiler over time for different agents.

Table 3: Rust compiler edge coverage for all generated programs and the compilable subset. Chart-agent results are medians of three five-day runs. Bold values indicate the highest coverage.

Generator	All programs	Compilable only
SYNSEMA (chart)	148,685	71,689
DeepSeek	**180,236**	**125,809**
Gemma	142,492	93,649

Table 4: Median negative log-likelihood (NLL, nats/token) of Rust programs under Qwen3-8B-Base, grouped by source-token count. Bold values indicate the highest NLL in each column.

Programs	<16	16–31	32–63	64–127	128–255	256–511
SYNSEMA (compilable)	6.11	4.71	3.57	2.51	1.85	1.33
DeepSeek	–	–	–	1.87	0.92	0.69
Gemma	–	–	–	–	0.74	0.63
SYNSEMA (crash/timeout)	**7.31**	**5.62**	**4.16**	**3.10**	**1.91**	**1.58**
Rust regression tests	5.52	3.67	2.26	1.62	1.16	0.76

Bug discovery. We collect all programs for which the compiler crashes, *i.e.* prematurely aborts compilation due to some unexpected failure condition. These programs are particularly valuable, because these are rare edge-cases which the compiler developers did not consider, generally difficult to find. Using SYNSEMA, we discovered 14 *distinct* crashes and hangs (12 of `rustc` and 2 of `javac`): 5 previously unknown, 6 previously known, and 3 currently under investigation. All but two of these crashes and hangs could be found with the chart agents. It is worth noting that the baseline triggered no such crashes or hangs in our experiment, demonstrating the usefulness of SYNSEMA.

We examine the feature combinations in four reduced Rust reproducers. In Figure 6a (#143413), a `for` binder contains a `const` parameter whose type uses an elided reference lifetime in `c(&())`. The compiler lowers the parameter before creating the extra lifetime's definition, causing an internal compiler error (ICE). Figure 6b (#146084) combines a `const` parameter of unsized slice type `[()]` with a diverging default expression `{ loop {} }`. Evaluating this ill-formed default reaches an interpreter assertion requiring a sized layout. In Figure 6c (#146754), an `async` function declaration inside an `extern` block uses a closure as the array-length expression in its return type. The compiler diagnoses the invalid `async` qualifier and array length, but then crashes when looking up the missing internal node for the opaque return type introduced for `async`. In Figure 6d (#146767), a macro matcher nests a comma-separated repetition of an empty token sequence inside a zero-or-more repetition; invoking the macro with empty input hangs the compiler.

Comparison with pretrained LLMs. To assess how learning from compiler feedback compares with using pretrained LLMs, we compare SYNSEMA with Fuzz4All [34], an LLM-based fuzzer

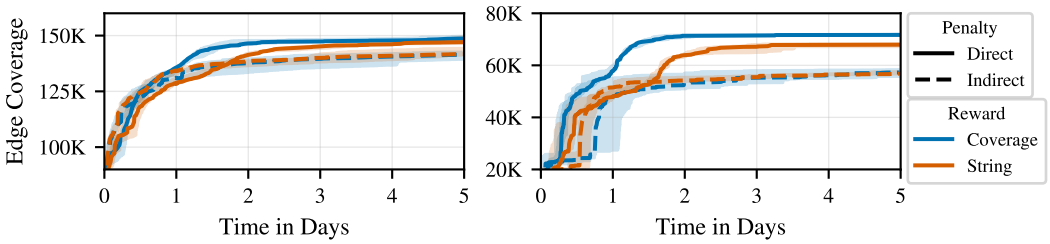


Figure 5: Ablation of different reward configurations for all generated programs (left) and only semantically valid ones (right). Colors indicate the reward type, *i.e.*, coverage- (blue) or string-based (orange), while line styles indicate whether the reward is direct (solid) or indirect (dashed).

shown to be effective at finding compiler bugs. We run one generation campaign per model on Rust, using DeepSeek-V4-Pro as a robust large model and Gemma-3-4B-IT as a small, fast alternative. We allocate USD 100 per model, comparable to the approximate GPU-rental cost of one five-day SYNSEMA run. DeepSeek exhausted its budget, while the cheaper Gemma ran for four days before we stopped it, having spent only USD 40.19. We measure coverage and compiler acceptance using the same instrumented Rust compiler as for SYNSEMA. DeepSeek generated 29,135 programs, of which 19,403 compiled and 27,097 were unique. Gemma generated 404,972 programs, of which 379,738 compiled and 26,653 were unique. Table 3 reports the compiler edge coverage achieved by these programs. DeepSeek achieves higher coverage than the chart agent for both all programs and compilable programs, while Gemma exceeds it only for compilable programs. However, DeepSeek triggered only one compiler crash and Gemma found no reproducible compiler bugs, compared with the 12 Rust compiler bugs found by SYNSEMA across configurations and runs. These results demonstrate that SYNSEMA can expose compiler bugs with limited compute and no pretraining, and that higher edge coverage alone need not translate into more bug discoveries.

Because both LLMs generated over 99 % syntactically valid programs without grammar constraints, there is little room to improve syntactic validity by adding constrained decoding. We expect very similar results with grammar-constrained decoding.

Common versus uncommon programs. We hypothesize that the LLMs' limited bug-finding effectiveness may stem from generating common programs. To investigate this hypothesis, we score compilable Rust outputs from SYNSEMA, Gemma, and DeepSeek, together with SYNSEMA's crash- or timeout-triggering inputs and Rust compiler regression tests. The SYNSEMA corpora pool the top-down, bottom-up, and chart agents. We use the pretrained *base* Qwen3-8B model, which did not generate any of these programs, as a proxy for commonness. For each program, we compute its negative log-likelihood (NLL): the average negative log-probability the model assigns to each source token given the preceding tokens, measured in nats/token. Higher NLL means that the model finds the program less likely. The full procedure is described in Appendix A.7.

Table 4 shows that SYNSEMA's compilable programs have higher median NLL than the LLM outputs and Rust regression tests in every reported comparison within a length range. Its crash- or timeout-triggering inputs have still higher NLL. These results provide preliminary evidence that SYNSEMA explores less common regions of the program space. Unlike pretrained LLMs, SYNSEMA starts from a randomly initialized model and learns from compiler feedback without a seed corpus. Its exploration is therefore not guided by the distribution of developer-written programs which may help it discover unusual feature combinations that compiler developers did not anticipate.

4.3 Ablation Study of the Reward Mechanism

We ablate our fine-grained semantic feedback (penalties) and the coverage rewards with Rust and the chart parser. We propose an alternative design for each part: (1) instead of *directly* leveraging the compiler's source spans and mapping the penalty back to the specific actions, we *indirectly* assign the reward to the last action and (2) instead of using the *coverage* information to compute the novelty score, we give each compilable *program string* a reward of 10 the first time it is generated and linearly decay this to 0 by the time it was generated 1000 times. Figure 5 shows that the direct semantic feedback has the largest effect, increasing edge coverage by 25.2 % over the undirected feedback

when considering only semantically valid programs. Coverage feedback also improves coverage but has a smaller effect, contributing an additional 3.2 % edges on average. Coverage rewards encourage *semantic* diversity, while string uniqueness encourages *syntactic* diversity. Although SYNSEMA consistently achieves higher coverage, we hypothesize that string uniqueness reasonably approximates coverage rewards when generating small programs for compiler testing.

5 Related Work

Learning-based program generation. LLMs have been prompted to generate test programs [34; 36], while Montage [17] learns to recombine program fragments. Earlier work framed testing as reinforcement learning but focused on mutation decisions rather than directed semantic learning [5; 10; 18; 19]. These approaches normally rely on pretrained models, but they do not improve the model for semantic understanding, whereas SYNSEMA continuously learns from compiler feedback. Related top-down representations appear in semantic parsing [24], code generation [29; 30], program synthesis [23; 35], and transformer pretraining [39]; these works and RL with chart parsing for program synthesis [6] pursue different objectives without systematically comparing these action spaces for compiler testing. GrammarCoder [20] uses AST-derived representations similar to our top-down derivations for continued LLM pretraining and instruction tuning, whereas SYNSEMA explores parser-derived action spaces with compiler-feedback RL. To our knowledge, SYNSEMA is the first to systematically design and evaluate top-down, bottom-up, and chart grammar processing algorithms for learning semantic constraints, and to demonstrate their utility in compiler testing (Section 4.2) beyond dataset-oriented evaluation as in existing works.

Rule-based program generation. Coverage-guided testing tools [26; 38] apply random mutations to produce new input programs but, unlike SYNSEMA, cannot reliably generate syntactically valid programs. Grammar-based generators [4; 28; 32] enforce syntactic correctness through grammar-guided mutations but typically disregard semantics. Semantic-centric generators [8] encode semantic constraints; Polyglot [7], Csmith, RustSmith, and Rustlantis [27; 33; 37] use manually crafted semantic rules or generation logic. CodeAlchemist [12] assembles seed fragments using semantic-aware mechanisms, while Zest [21] augments syntax-aware testing tools with semantic feedback. SYNSEMA instead learns semantics from compiler feedback with a grammar constraining its actions.

6 Conclusion

This paper introduces SYNSEMA, a novel approach that reformulates input generation as a syntax-guided reinforcement learning task. By constraining the action space with grammars, we separate syntactic and semantic concerns, allowing an agent to focus on learning semantics in a much smaller space. Our empirical evaluation of the Rust and Java compilers demonstrates that syntax-guided approaches achieve semantic correctness rates much higher than those of token-based baselines and discover five previously unknown bugs. To improve semantic understanding and extend program-generation capabilities, future work could explore rewards beyond compiler feedback.

Limitations. First, our approach does not yet support comments in the generated programs, as we focus on semantic correctness rather than code readability. Second, we did not adopt any pretrained model. Unlike token sequences, the operation-based representations (LL productions and LR shift/reduce actions) have, to our knowledge, no existing pre-trained models. Third, the current prototype is restricted to generating relatively small programs and relies on a predefined subword vocabulary. Although this design is demonstrably effective for generating rare programs specifically for compiler testing, its effectiveness in other scenarios has not yet been validated. Fourth, due to resource constraints, the evaluation was conducted over only five days with the 124M model. Finally, compiler acceptance does not guarantee full specification conformance or runtime correctness. Since the compiler under test also serves as our semantic oracle, detecting incorrect acceptance of semantically invalid programs without other observable failures would require additional measures, *e.g.*, differential testing across compiler implementations or versions.

Acknowledgments and Disclosure of Funding

We thank the anonymous reviewers for their valuable feedback on an earlier version of this paper. This work was partially supported by a research award from the Hasler Foundation, which we appreciate and acknowledge. The RL experiments and NLL scoring used ETH Zurich's Euler cluster.

References

[1] Santiago Acevedo, Alessandro Laio, and Marco Baroni. Differential syntactic and semantic encoding in llms. *CoRR*, abs/2601.04765, 2026.

[2] Alfred V Aho and Jeffrey D Ullman. *The theory of parsing, translation, and compiling*, volume 1. Prentice-Hall Englewood Cliffs, NJ, 1972.

[3] ANTLR. Grammars V4 (Antlr), 2013. URL https://github.com/antlr/grammars-v4. Accessed: 2026-04-28.

[4] Cornelius Aschermann, Tommaso Frassetto, Thorsten Holz, Patrick Jauernig, Ahmad-Reza Sadeghi, and Daniel Teuchert. NAUTILUS: fishing for deep bugs with grammars. In *26th Annual Network and Distributed System Security Symposium (NDSS)*, 2019.

[5] Konstantin Böttinger, Patrice Godefroid, and Rishabh Singh. Deep reinforcement fuzzing. In *IEEE Security and Privacy Workshops*, pages 116–122, 2018.

[6] Rudy Bunel, Matthew J. Hausknecht, Jacob Devlin, Rishabh Singh, and Pushmeet Kohli. Leveraging grammar and reinforcement learning for neural program synthesis. In *6th International Conference on Learning Representations (ICLR)*, 2018.

[7] Yongheng Chen, Rui Zhong, Hong Hu, Hangfan Zhang, Yupeng Yang, Dinghao Wu, and Wenke Lee. One engine to fuzz 'em all: Generic language processor testing with semantic validation. In *42nd IEEE Symposium on Security and Privacy (S&P)*, pages 642–658, 2021.

[8] Kavya Chopra, Cong Li, Thodoris Sotiropoulos, and Zhendong Su. Semantic reification: A new paradigm for random program generation. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2026.

[9] Jay Earley. An efficient context-free parsing algorithm. *Commun. ACM*, 13(2):94–102, 1970.

[10] Jueon Eom, Seyeon Jeong, and Taekyoung Kwon. Fuzzing JavaScript interpreters with coverage-guided reinforcement learning for LLM-based mutation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 1656–1668, 2024.

[11] James Gosling, Bill Joy, Guy Steele, Gilad Bracha, Alex Buckley, Daniel Smith, and Gavin Bierman. *The Java Language Specification: Java SE 26 Edition*. Oracle America, Inc., 2026.

[12] HyungSeok Han, DongHyeon Oh, and Sang Kil Cha. CodeAlchemist: Semantics-aware code generation to find vulnerabilities in JavaScript engines. In *26th Annual Network and Distributed System Security Symposium (NDSS)*, 2019.

[13] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to automata theory, languages, and computation, 3rd Edition*. Addison-Wesley, 2007.

[14] Dan Horgan, John Quan, David Budden, Gabriel Barth-Maron, Matteo Hessel, Hado van Hasselt, and David Silver. Distributed prioritized experience replay. In *6th International Conference on Learning Representations (ICLR)*, 2018.

[15] Leslie Pack Kaelbling, Michael L. Littman, and Andrew W. Moore. Reinforcement learning: A survey. *J. Artif. Intell. Res.*, 4:237–285, 1996.

[16] Donald E. Knuth. On the translation of languages from left to right. *Inf. Control.*, 8:607–639, 1965.

[17] Suyoung Lee, HyungSeok Han, Sang Kil Cha, and Sooel Son. Montage: A neural network language model-guided JavaScript engine fuzzer. In *29th USENIX Security Symposium*, pages 2613–2630, 2020.

[18] Xiaoting Li, Xiao Liu, Lingwei Chen, Rupesh Prajapati, and Dinghao Wu. ALPHAPROG: reinforcement generation of valid programs for compiler fuzzing. In *Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI)*, pages 12559–12565, 2022.

[19] Xiaoting Li, Xiao Liu, Lingwei Chen, Rupesh Prajapati, and Dinghao Wu. FuzzBoost: Reinforcement compiler fuzzing. In *24th International Conference on Information and Communications Security (ICICS)*, pages 359–375, 2022.

[20] Qingyuan Liang, Zhao Zhang, Zeyu Sun, Zheng Lin, Qi Luo, Yueyi Xiao, Yizhou Chen, Yuqun Zhang, Haotian Zhang, Lu Zhang, Bin Chen, and Yingfei Xiong. Grammar-based code representation: Is it a worthy pursuit for LLMs? In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 15640–15653, 2025.

[21] Rohan Padhye, Caroline Lemieux, Koushik Sen, Mike Papadakis, and Yves Le Traon. Semantic fuzzing with Zest. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 329–340, 2019.

[22] Terence Parr. *The definitive ANTLR 4 reference*. The Pragmatic Bookshelf, 2013.

[23] Julian Parsert and Elizabeth Polgreen. Reinforcement learning and data-generation for syntax-guided synthesis. In Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan, editors, *Thirty-Eighth AAAI Conference on Artificial Intelligence*, pages 10670–10678, 2024.

[24] Maxim Rabinovich, Mitchell Stern, and Dan Klein. Abstract syntax networks for code generation and semantic parsing. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1139–1149, 2017.

[25] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In Katrin Erk and Noah A. Smith, editors, *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016.

[26] Kosta Serebryany. Continuous fuzzing with libfuzzer and addresssanitizer. In *IEEE Cybersecurity Development (SecDev)*, page 157, 2016.

[27] Mayank Sharma, Pingshi Yu, and Alastair F. Donaldson. RustSmith: Random differential compiler testing for Rust. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 1483–1486, 2023.

[28] Prashast Srivastava and Mathias Payer. Gramatron: effective grammar-aware fuzzing. In *30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 244–256, 2021.

[29] Zeyu Sun, Qihao Zhu, Lili Mou, Yingfei Xiong, Ge Li, and Lu Zhang. A grammar-based structural CNN decoder for code generation. In *The 33rd AAAI Conference on Artificial Intelligence (AAAI)*, pages 7055–7062, 2019.

[30] Zeyu Sun, Qihao Zhu, Yingfei Xiong, Yican Sun, Lili Mou, and Lu Zhang. TreeGen: A tree-based transformer architecture for code generation. In *The 34th AAAI Conference on Artificial Intelligence (AAAI)*, pages 8984–8991, 2020.

[31] Vince Szabo, Dominik Winterer, and Zhendong Su. Compilation quotient (CQ): A metric for the compilation hardness of programming languages. *CoRR*, abs/2406.04778, 2024.

[32] Junjie Wang, Bihuan Chen, Lei Wei, and Yang Liu. Superion: grammar-aware greybox fuzzing. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 724–735, 2019.

[33] Qian Wang and Ralf Jung. Rustlantis: Randomized differential testing of the Rust compiler. *Proc. ACM Program. Lang.*, 8(OOPSLA2):1955–1981, 2024.

[34] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 126:1–126:13, 2024.

[35] Yingfei Xiong and Bo Wang. L2S: A framework for synthesizing the most probable program under a specification. *ACM Trans. Softw. Eng. Methodol.*, 31:34:1–34:45, 2022.

[36] Wenzhang Yang, Cuifeng Gao, Xiaoyuan Liu, Yuekang Li, and Yinxing Xue. Rust-twins: Automatic Rust compiler testing through program mutation and dual macros generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 631–642, 2024.

[37] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 283–294, 2011.

[38] Michał Zalewski. American Fuzzy Lop, 2014. URL http://lcamtuf.coredump.cx/afl/.

[39] Qihao Zhu, Qingyuan Liang, Zeyu Sun, Yingfei Xiong, Lu Zhang, and Shengyu Cheng. GrammarT5: Grammar-integrated pretrained encoder-decoder neural model for code. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 76:1–76:13, 2024.

A Appendix

A.1 Parsing-Specific Program Generation Algorithms

This section presents the parser-specific algorithms that the actor uses for program generation (Section 3.1), including those for the top-down agent (Algorithm 1), bottom-up agent (Algorithm 2), and the chart agent (Algorithm 3).

Algorithm 1: Program Generation (Top-Down)

Input: Grammar $G = (N, T, P, S)$
$\alpha \leftarrow S$;
while α contains non-terminals **do**
  $nt \leftarrow$ leftmost non-terminal in α ;
  $action \leftarrow$ `sample`$(\{p \in P : \texttt{lhs}(p) = nt\})$;
  $pos \leftarrow$ leftmost position of nt in α ;
  $\alpha \leftarrow$ replace nt with `rhs(`*action*`)` at pos in α ;
end
return `join(`α`)`

Algorithm 2: Program Generation (Bottom-Up)

Input: LR(1) parsing table T
$S \leftarrow$ [start state of T], $\alpha \leftarrow \varepsilon$;
repeat
  state $\leftarrow$ top of S ;
  action $\leftarrow$ `sample(`actions in *T* for *state*`)` ;
  if `isShift(`*action*`)` **then**
    push `nextState(`*action, T*`)` onto S ;
    append `terminalOf(`*action*`)` to α ;
  else if `isReduce(`*action*`)` **then**
    pop |`production(`*action*`)`| elements from S ;
    push `goto(`*top of S, T*`)` onto S;
until `isAccept(`*action*`)`;
return `join(`α`)`

Algorithm 3: Program Generation (Chart)

Input: Terminal vocabulary T, Grammar G
$\alpha \leftarrow \varepsilon$;
repeat
  valid $\leftarrow$ `Earley`(G, α) ; `// Get valid terminals for prefix` α
  $t \leftarrow$ `sample`$(valid)$;
  if $t \neq$ *EOS* **then** append t to α ;
until $t =$ *EOS*;
return `join(`α`)`

A.2 Action-Space Examples and Alternatives

Top-down example. For the CFG in Figure 1b, the agent has four actions, one per production. The first row of Table 1 shows the top-down action sequence for `NUM+NUM*NUM`; when the current action sequence is $[E \rightarrow E\ O\ E, E \rightarrow$ `NUM`$]$, the valid actions are constrained into the two *Op* productions, *i.e.*, (iii) and (iv) in Figure 1b, according to the LL algorithm.

Bottom-up example. The second row of Table 1 shows the action sequence for the example program `NUM+NUM*NUM`, with green nodes indicating shift actions and red nodes indicating reduce actions. The LR automaton of our example CFG, which constrains the action space, is displayed in Appendix A.3.

Chart example and representation alternatives. The third row of Table 1 shows the resulting action sequence for the example `NUM+NUM*NUM`, which is the program itself. Earley parsing determines that,

given the action sequence $[\texttt{NUM}, +]$, the only valid next action is the terminal `NUM`, thereby constraining the action space from three possible terminals to a single valid choice.

Note that top-down and bottom-up parsing can also be implemented with token-based representations; however, this would be equivalent to using an Earley parser. Similarly, an Earley parser can be implemented with operation-based representations, but this is empirically suboptimal: in a five-minute Rust generation experiment, producing 661,600 tokens required 144,773,493 actions (about 218.8 actions per token), indicating substantial overhead relative to token-based representations.

A.3 LR Automaton of the Example CFG

In the LR algorithm, an automaton through which our bottom-up agents will traverse is usually expressed via Action-Goto tables, such as Table 5 for our example CFG in Figure 1b. LR(1) Action tables can contain conflicts that need resolution during parsing. During generation, the agent resolves them dynamically by selecting an action without a deterministic lookahead choice.

Table 5: LR(1) parsing table for the grammar in Figure 1b ("$" means EOS).

	Action (Terminals)				Goto (Nonterminals)	
State	`NUM`	`+`	`*`	`$`	*Expr*	*Op*
1	S_2				3	
2		R_2	R_2	R_2		
3		S_6	S_5	A		4
4	S_2				7	
5	R_4					
6	R_3					
7		S_6/R_1	S_5/R_1	R_1		4

A.4 Hyperparameters for Empirical Evaluation

Table 6 presents the default hyperparameters for the RL agent and Ape-X [14] used in our experiments.

Table 6: Hyperparameters used for empirical evaluation.

Parameter	**Value**
Model Architecture	
Embedding dimension d_{embd}	768
Layers L / Heads H	12 / 12
Context length T	256
Dropout	0.05
Optimisation	
Optimiser	AdamW
Learning rate	10^{-3}
$(\beta_1,\ \beta_2)$	$(0.9,\ 0.95)$
Weight decay	0.1
Batch size	32
RL Algorithm	
Discount factor γ	0.99
n-step returns	1
Target network update freq.	500
Value rescaling ε	10^{-3}
Priority mixing η	0.9
Actor update interval	500
Number of actors	32
Base exploration ε_0	0.4
Exploration scale α	3.0
Maximum episode length	128

Parameter	**Value**
Replay Buffer	
Capacity	500,000
Success/failure sample ratio	0.5
Uniform random sample rate	0.1
Reward Function	
Per-step cost	-5×10^{-4}
Max success reward	$+10$
Error penalty	-1 per diagnostic

A.5 Learned versus Random Policies

We conduct two analyses to assess whether our agents learn semantic constraints. First, we examine compilable Rust programs for nontrivial language features. Second, for each action representation, we replace the learned policy with a random one, isolating policy learning from syntax enforcement alone. Table 7 reports the fraction of compilable programs containing each feature under the learned and random policies, summed over three runs.

All three SYNSEMA agents exercise every feature, though with varying frequencies. Macros are most prevalent, as fewer semantic constraints apply to them. Lifetimes, generics, and unsafe occur in $0.98\,\%$ to $9.67\,\%$ of programs, while ownership, trait bounds, const generics, and higher-ranked lifetimes/trait bounds are rarer, from $0.001\,\%$ to $2.74\,\%$. The random-policy comparison provides additional evidence that this diversity comes from the learned policy, not from syntax enforcement alone: the learned policy has higher reported frequencies for every non-macro feature, sometimes by orders of magnitude. Macros are the sole exception, and the learned policy generates them less often in two of three representations. SYNSEMA thus generates programs with diverse semantic features, not merely higher compiler acceptance.

Table 7: Feature frequencies. < 0.01 denotes a positive frequency rounding to 0.00.

Feature	**Baseline** (%)		**Top-Down** (%)		**Bottom-Up** (%)		**Chart** (%)	
	Learned	Random	Learned	Random	Learned	Random	Learned	Random
Ownership/borrowing	0.00	n/a	0.43	< 0.01	0.17	< 0.01	0.20	0.00
Lifetimes	0.00	n/a	5.31	1.87	1.79	0.01	8.35	0.14
Trait bounds	0.00	n/a	< 0.01	0.00	< 0.01	0.00	0.02	0.00
Generics	0.00	n/a	1.75	0.02	0.98	0.01	9.67	0.20
Macros	0.00	n/a	76.90	81.06	91.87	98.44	74.48	62.43
Const generics	0.00	n/a	0.16	0.00	0.06	0.00	0.27	0.00
Unsafe	0.00	n/a	8.53	0.78	2.55	0.05	6.10	0.54
Higher-ranked	0.00	n/a	2.74	0.07	0.45	< 0.01	0.18	0.00
Learned-policy n	336		1,396,268		2,284,384		2,118,690	

A.6 Reduced Rust Bug Reproducers

```
type A = for<const B: c(&())> D;
```

(a) ICE due to incorrect lifetime lowering.

```
struct S<const N: [()] = { loop {} }>;
```

(b) ICE due to ill-formed `const` generics.

```
unsafe extern "C" {
    async fn function() -> [(); || {}];
}
```

(c) ICE with a closure in an `async` foreign function.

```
macro_rules! m { [ $($(),+)* ] => {} }

m! {}
```

(d) Hang with nested macro repetitions.

Figure 6: Four Rust bugs (reduced) discovered by SYNSEMA and submitted to the developers.

A.7 Measuring Program Commonness

Before scoring, we remove comments, normalize identifiers, and format programs. To prevent arbitrary token sequences in unexpanded macro bodies from inflating NLL, we filter repetitive programs by discarding those with at least twelve lexical tokens if one token accounts for more than $30\,\%$ of them. We score source tokens without a prompt, using an unscored initial EOS token. We form length buckets with the scorer's tokenizer and omit those with fewer than ten inputs per corpus.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The main claims made in the abstract and introduction accurately reflect the paper's contributions and scope.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The paper includes limitations in Section 6.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper does not include theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: We specify our approach and implementation (Section 3), evaluation setup (Section 4), and hyperparameters (Appendix A.4) in the paper. Code: https://github.com/levinwinter/synsema; data: https://doi.org/10.5281/zenodo.20339950.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
     (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: Code: https://github.com/levinwinter/synsema; data: https://doi.org/10.5281/zenodo.20339950.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The paper specified these information in Section 4 and in the code.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: The paper includes error bars in the reported results in Section 4.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).

- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: The paper provides these information in Section 4.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: We have read and understood the NeurIPS Code of Ethics and have made every effort to conform to it.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: This paper contributes to a better understanding of the RL paradigm in generating programs for compiler testing, with grammar guidance. It does not impact society at large.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not release new models. The dataset is a simple collection of programs, which does not pose any risks.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: The paper does not use any datasets. We adhere to the licenses and terms of use of the GPT-2 implementation, Fuzz4All, DeepSeek-V4-Pro, Gemma-3-4B-IT, and Qwen3-8B-Base.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: Our code is documented with a comprehensive README.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.

- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper does not describe research involving crowdsourcing or human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper does not describe research involving crowdsourcing or human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: Our core method does not use pretrained LLMs. Their use is limited to evaluation baselines and program-likelihood scoring.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.